A sensor reading over MQTT

I2C sensor in TinyGo, MQTT publishing, live value in Flutter

Dinu-Ștefan Rusu MEC

Lab objectives

Read a sensor and publish it

Drive an I2C sensor from TinyGo with a driver from the TinyGo drivers repository, then publish the reading as JSON to your Mosquitto broker every two seconds.

Show it live in the app

Subscribe with mqtt_client, push every message into a BLoC, render it in a BlocBuilder, and survive the broker or the board going away.

Time
2 hours

Present
To the lab instructor, at the end of this lab

Hardware setups and message contract

ESP32-C3 or ESP32-S3

Wi-Fi comes from the espradio package. The board publishes to the broker itself.

Raspberry Pi Pico

No Wi-Fi on the plain Pico. It prints the reading, and a laptop bridge forwards it.

Wokwi simulator

No physical sensor, no route to your laptop. Use a simulated part or a counter, and a public test broker.

The contract

All three paths give the broker the same topic, the same JSON, one message every two seconds.

The broker is the Mosquitto you installed in Lab 1. Start it before anything else.

Dependencies and file layout

1. Add the dependencies:

```
cd device && go get tinygo.org/x/drivers github.com/soypat/natiu-mqtt
cd ../app && flutter pub add mqtt_client
```

2. Firmware in `device/lab2/main.go`. App code in `lib/lab2/`.
3. Leave a subscriber running in a terminal for the whole lab. It is your ground truth.

Order of work

Sensor first, then the broker, then the app. Do not write Flutter code before a reading appears in `mosquitto_sub`.

Read one sensor over I2C

1. Pick a sensor with a driver in `tinygo.org/x/drivers`. The BME280 is cheap, common, and reads temperature.
2. Wire it to the two I2C pins of your board, supply and ground.
3. Configure the bus at 400 kHz, then the driver.
4. Print one reading every two seconds, and read it back with `tinygo monitor`.

Done when

- [ ] The serial line shows a new value every two seconds.
- [ ] The value changes when you breathe on the sensor.
- [ ] A read error prints a message instead of stopping the loop.

Sensor loop

```
// tinygo.org/x/drivers/bme280
i2c := machine.I2C0
i2c.Configure(machine.I2CConfig{
    Frequency: 400 * machine.KHz})
s := bme280.New(i2c)  // 0x76 or 0x77
s.Configure()

for {
    mC, err := s.ReadTemperature()
    if err != nil { println(err.Error()) }
    println("c:", float32(mC)/1000)
    time.Sleep(2 * time.Second)
}
```

Milli-degrees, not degrees.
The drivers return thousandths of a degree Celsius. Divide once and keep one unit in your code.

Watch out

`Connected()` answers before the first read. Call it and print the answer: a sensor that never acknowledges is a wiring fault.

Simulated reading without a sensor

```
// Same program, one line swapped. Everything downstream stays identical.
celsius := 20 + float32(seq%100)/10   // a ramp, not a constant
seq++
```

On Wokwi

Use the board Lab 1 named for you. If the part library carries an I2C sensor with a TinyGo driver, wire it in the diagram. If not, feed the loop the counter above and say so when you present.

The broker, from Wokwi

The simulator runs in the browser, so a broker on your own machine is out of reach. Publish to a public test broker, on a topic prefix nobody else will use.

Publish a value that moves, so the number on the phone shows the chain is alive.

Publish the reading over MQTT

1. Join the network. On the ESP32-C3 and ESP32-S3 the radio comes from the espradio package, not from the standard library.
2. Connect to the broker with a client id that carries your name.
3. Publish one JSON document on `course/<name>/reading` every two seconds, with QoS 1 and the retain flag off.
4. Print the result of every step.

Done when

- [ ] `mosquitto_sub` prints one line every two seconds.
- [ ] The payload parses as JSON and carries a sequence number.
- [ ] Killing the broker and starting it again brings the board back without a reflash.

Wi-Fi and broker connection

```
lk := link.Esplink{}   // espradio
netdev.UseNetdev(&lk)
lk.NetConnect(&nl.ConnectParams{
    Ssid: ssid, Passphrase: pass})

conn, _ := net.Dial("tcp", broker)
var q mqtt.OrchestratorNoAlloc
q.Configure(qcfg)   // QoS 1 state
c := mqtt.NewClient(mqtt.ClientConfig{
    Orchestrator: &q})
var v mqtt.VariablesConnect
v.SetDefaultMQTT([]byte(id))
c.Connect(ctx, conn, &v)
```

Treat this as a shape.
The radio is `tinygo.org/x/espradio`, the client is `natiu-mqtt`. Keep the sequence: join, dial, connect, publish, check the error each time.

The address

`broker` is your laptop's address and port on the lab network, such as `192.168.1.10:1883`. Not `localhost`: that is the board.

Topic and payload

```
topic := "course/" + name + "/reading"       // one student, one topic
p := fmt.Sprintf(`{"name":%q,"c":%.2f,"seq":%d}`, name, celsius, seq)
fl, _ := mqtt.NewPublishFlags(mqtt.QoS1, false, false)   // retain off
pv := mqtt.VariablesPublish{TopicName: []byte(topic)}
c.PublishPayload(fl, pv, []byte(p))  // then HandleNext for the PUBACK
time.Sleep(2 * time.Second)
```

Rules

Identity in the topic, data in the payload. Never publish the value as a topic level.

The sequence number is yours, not the protocol's: it is the only way to see a gap.

Why QoS 1

At QoS 1 the broker acknowledges each message, so a lost packet is resent. A duplicate is harmless here because every document carries its own sequence number.

Pico without Wi-Fi: laptop bridge

```
# macOS: stty -f <port> 115200      Linux: stty -F <port> 115200
cat /dev/tty.usbmodem1101 | while IFS= read -r line; do
  mosquitto_pub -h localhost -q 1 -t course/t07/reading -m "$line"
done
```

The board prints the JSON document, one per line, as a networked board would publish it. The laptop only forwards. Nothing else in the lab changes: the phone cannot tell the two set-ups apart.

Watch out

`tinygo monitor` holds the serial port open. Stop it before you start the bridge.

Show the value live in the app

1. Add a `ReadingBloc` with one event for an incoming document and a state holding the last value and its sequence number.
2. Connect an `MqttServerClient` and subscribe to your topic at QoS 1.
3. Decode each message on the updates stream and add an event to the BLoC.
4. Render the value in a `BlocBuilder`, with a visible waiting state before the first message.
5. Reconnect with a growing delay when the connection drops.

Done when

- [ ] The number on screen changes every two seconds.
- [ ] Unplugging the board stops the updates and the screen says so.
- [ ] Plugging it back in resumes updates with no restart of the app.
- [ ] The client is closed when the screen is disposed.

Connect, subscribe, decode

```
final c = MqttServerClient('10.0.2.2', 'phone-$name')  // mqtt_client
  ..port = 1883
  ..keepAlivePeriod = 30
  ..onDisconnected = _scheduleReconnect;
await c.connect();
c.subscribe('course/$name/reading', MqttQos.atLeastOnce);
c.updates!.listen((events) {
  final m = events.first.payload as MqttPublishMessage;
  final text = MqttPublishPayload.bytesToStringAsString(m.payload.message);
  _bloc.add(ReadingReceived(jsonDecode(text)));
});
```

`10.0.2.2` is your machine from the Android emulator. A real phone uses the laptop's address.

Rendering and reconnecting

```
BlocBuilder<ReadingBloc, ReadingState>(
  builder: (ctx, s) => Text(s.label),   // 'waiting' or '21.4 C'
)
void _scheduleReconnect() async {
  var delay = const Duration(seconds: 1);
  while (!_closed && !_connected) {
    await Future.delayed(delay);
    try { await c.connect(); } catch (_) {}
    delay *= 2;              // stop growing at a minute
  }
}
```

Two clients with the same id disconnect each other, which looks like a broken network.

Acceptance checks

```
mosquitto_sub -h localhost -q 1 -t 'course/+/reading' -v
```

The demo

- [ ] The topic prints one document every two seconds.
- [ ] The app shows the same value, live.
- [ ] Pulling the board’s cable stops the updates.
- [ ] Plugging it back in resumes them, with no app restart.

Be ready to explain

The firmware and the app code, line by line.

Your topic, your board or simulator, and the broker address you used.

Common errors

`Connected() is false`	Wrong I2C address: scan the bus, `0x76` or `0x77`.
`The bus scan prints nothing`	Nothing acknowledged. Check supply, ground, and the two I2C pins.
`Error: Connection refused`	Nothing listens on 1883. Start Mosquitto, bind the network address.
`Both clients reconnect in a loop`	One client id, two clients. Give each of them its own id.
`CLEARTEXT not permitted`	Android blocks a plain broker. Set usesCleartextTraffic true in the manifest.

Presentation

The demo, live · your topic, explained